CircuitPilot: An Agent-Operable Toolchain from Natural Language to Manufacturable PCBs

Abdelhay Ali

***Abstract*—Large language models (LLMs) can read datasheets, choose components and reason about circuits, yet they remain poor at the one thing a printed circuit board (PCB) ultimately is: exact geometry that must pass manufacturing rules. Existing LLM-for-EDA work concentrates on hardware description languages and integrated-circuit flows, and recent PCB efforts address isolated sub-tasks such as schematic generation or benchmark environments. We present *CircuitPilot*, a browser-based schematic-to-fabrication toolchain built around one principle: *the language model decides, the geometric engines guarantee*. The agent expresses intent through 105 typed tools exposed over the Model Context Protocol (MCP) and an in-app copilot, while deterministic engines—a negotiated-congestion grid router with blocker discovery, an exact-geometry design-rule checker, distance-field copper pours, a desktop-CNC isolation-milling generator with automatic toolpath verification, a schematic organizer with an orthogonal wire router, and a 3D enclosure kernel—produce outputs that are correct by construction or reported with an actionable reason. One JavaScript engine runs unchanged in the browser, a static web deployment, a Windows application and a headless MCP server. On 15 real boards from a wearable biomedical research project (608 parts in total), negotiated rip-up raised fully-routed boards from 7 to 11 and routed 455 of 468 nets with a median run time of 25.2 s; compact tool answers cut the state an agent must read to 7 % of the raw project for the PCB layout. Isolation-milling jobs for single-sided boards were generated with zero detected shorts or cut nets. We describe the architecture, the algorithms, the agent-oriented tool design, and the limitations that remain.**

***Index Terms*—Electronic design automation, printed circuit board, large language model agents, Model Context Protocol, maze routing, rip-up and reroute, isolation milling, schematic layout.**

I. Introduction

A researcher who needs a small custom board—a sensor front end, a wearable node, a test fixture—faces a toolchain built for professional layout engineers. Capable EDA suites such as KiCad [1] are free, but the path from an idea to a board that a fabricator accepts, or that a desktop CNC machine can mill, still requires symbol and footprint management, placement intuition, routing skill, knowledge of design rules, and a separate computer-aided manufacturing (CAM) step [2], [3]. For research groups whose expertise is the application rather than the board, this overhead is repeated for every design iteration.

Large language models appear to be the missing assistant: they read datasheets, recall reference circuits and decompose tasks, and agent frameworks let them act through tools [4], [5]. LLM research in electronic design automation has, however, focused mainly on register-transfer-level code and chip design [6]–[11] and on analog circuit synthesis [12]. Work specific to PCBs has started only recently and targets individual stages: netlist-to-schematic conversion [13], schematic generation [14], question answering [15], and benchmark environments for layout [16], [17]. The central difficulty is unchanged: a PCB is a geometric object that must satisfy hundreds of clearance, width, drill and edge constraints exactly, and a language model that emits coordinates directly cannot be trusted to satisfy them.

Problem. How can a language-model agent take a design from a sentence to a manufacturable board—fabrication files for a PCB service, or G-code for a desktop CNC—without the model ever being responsible for geometric correctness, and without the user installing or learning a professional EDA tool?

Approach. We split responsibilities. The agent makes the decisions that need knowledge and judgment: which parts, how they connect, which nets are power, where connectors and antennas go, which design-rule preset suits the process. Deterministic engines turn those decisions into geometry and verify it. Every engine answers the agent in compact, structured text, and every failure carries a reason that names the obstacle and a next step. We call this principle *language decides, geometry guarantees*.

Motivating story. CircuitPilot grew out of a human-body-communication (HBC) electrocardiogram (ECG) wearable project in which one researcher needed dozens of board variants—transmitter and receiver nodes on different microcontrollers, coin-cell and round form factors, test panels, flexible electrode leads, and single-sided boards milled on a desktop CNC. The tool's own project database holds 65 such designs. Iterating at that rate with a conventional flow was impractical. The benchmark in Section V is drawn from these designs, so every result reflects a real board rather than a synthetic netlist.

Contributions.

1) *An agent-operable EDA architecture* in which a single JavaScript engine serves the browser editor, a static web deployment, a Windows application and a headless MCP [18], [19] server, exposing 105 typed tools (Section III).
2) *Six design principles for agent-facing EDA tools*: compact filtered state, actionable failure reasons, one-call recipes, background jobs, version safety and visual self-checks. We measure the first (Section III-A).

A. Ali, e-mail: abdelhay.ali.292@gmail.com.

Software: https://pcbgo.site, source: https://github.com/Abdelhayali/CircuitPilot.

3) *A negotiated-congestion grid router for PCBs* that combines windowed weighted A*, a bidirectional reachability filter, soft-mode blocker discovery, history costs, rip-up of the cheapest blockers, geometry straightening and exact-geometry repair (Algorithm 1).
4) *A verified desktop-CNC fabrication path*: isolation toolpaths from exact Euclidean distance transforms, an automatic check for shorts and cut nets, optional SMD-pad thermal relief, and 2D/3D previews of the milled board (Algorithm 2).
5) *A readable-schematic organizer* with an orthogonal maze wire router that keeps different nets from overlapping (Algorithm 3).
6) *An evaluation on 15 real boards*, with an ablation of rip-up and reroute, CNC statistics and tool-answer sizes. The benchmark scripts are released with the tool.

II. Related Work

Routing. Maze routing goes back to Lee's wave expansion [20], built on shortest-path search [21]. It was accelerated by Hadlock's detour-number search [22] and by heuristic A* search [23], while Hightower's line search [24] avoided the grid. Sequential routers depend on net order. Rip-up and reroute [25] and iterative global wiring [26] mitigate this, and PathFinder's negotiated congestion [27] turned it into a principled cost scheme; Kahng *et al.* [28] survey these methods. Freerouting [29] is the router most used with open PCB tools. Learning-based placement and routing have shown promise for chips [30], for PCB placement [31] and for circuit routing [32]. Our router borrows negotiation from PathFinder and adds blocker discovery and agent-readable failure explanations.

LLMs for EDA and PCBs. Surveys [10], [11] document rapid progress in HDL generation [6], [7], domain adaptation [8], conversational hardware design [9] and analog synthesis [12]. For boards, Schemato converts netlists to schematics [13], SchGen generates PCB schematics [14], PCB-QA evaluates PCB design knowledge [15], and PCBWorld and OmniLayout provide layout benchmarks [16], [17]. These works study individual stages or evaluate models; CircuitPilot is an end-to-end system in which the model never produces geometry itself.

Tool-using agents. ReAct interleaves reasoning and acting [4], and Toolformer learns when to call tools [5]. MCP standardizes how agents discover and call external tools [18], [19], and its security landscape has been surveyed [33]. We contribute design guidance for tools whose results are large geometric states.

Schematic drawing and milling. Automatic schematic generation dates to the 1980s [34] and draws on layered [35] and orthogonal [36] graph drawing. PCB isolation milling is established for prototyping [37], with FlatCAM [2] and pcb2gcode [3] converting Gerber files [38] into G-code as a separate step.

Table I positions CircuitPilot against these lines of work.

TABLE I: Scope of CircuitPilot relative to related lines of work.

	NL input	Sch-ematic	Place +route	Fab files (Gerber)	CNC G-code + enclosure
KiCad + Freerouting [1], [29]	–	✓	✓	✓	–
FlatCAM, pcb2gcode [2], [3]	–	–	–	input	G-code
LLM-for-HDL [6], [7]	✓	–	–	–	–
Schemato, SchGen [13], [14]	✓	✓	–	–	–
PCBWorld, OmniLayout [16], [17]	✓	bench.	bench.	–	–
CircuitPilot (ours)	✓	✓	✓	✓	✓

III. System Architecture

Fig. 1 shows the architecture. The design model is one JSON document: components with symbol, footprint and supplier data; named nets of `REF.PIN` references; the board outline; and the PCB state (placements, tracks, vias, pours, keep-outs, holes, text), together with design rules and the enclosure. Every editing operation is a function of the shared *engine* (about 540 kB of plain JavaScript, with no build step) that mutates this document inside a checkpointed transaction.

Hosts. The same engine files run in four hosts. (i) The browser app talks to a small Python server (projects and parts databases in SQLite, proxy to supplier APIs). (ii) A static deployment on GitHub Pages replaces the server with an in-browser backend: projects in IndexedDB, and part search through a stateless relay worker. (iii) A Windows application bundles the server. (iv) The MCP server loads the engine into a Node.js `vm` context and runs long operations (routing, optimisation, export) in worker threads, returning a job identifier that the agent polls. Edits made through MCP are saved to the same project store and appear live in an open browser tab, so a human can watch and intervene.

Agents. Two kinds of agent use the tools. External agents (Claude Desktop, Claude Code, any MCP client) call the 105 MCP tools. The in-app copilot uses the 88 engine tools with a model of the user's choice: Gemini [39], Claude, OpenAI-compatible endpoints, Ollama or LM Studio. In both cases the model never edits coordinates directly unless it calls an explicit geometric tool such as `add_track`, and even then the result is checked.

Parts. Part search covers the JLCPCB/LCSC catalogue, with symbols and footprints from the official EasyEDA library, plus KiCad footprints and a user library. `get_part` returns the exact pinout and footprint, so the agent wires pins by name instead of guessing.

A. Tool Interface

Table II groups the tools. They follow six principles derived from watching agents fail.

P1 Compact, filterable state. Answers are summaries rather than the raw document: `get_pcb_layout` lists parts and pads, filtered by reference or region; `run_drc` returns five violations unless `full` is set; `get_tracks` accepts `near` and `region`.

P2 Actionable failures. When a net cannot be routed, the router names the pads it could not join, what surrounds them (neighbouring pins, kept copper, keep-outs, the board edge) and whether a via fits beside the pad, and it suggests a next tool (Section IV-B).

P3 One-call recipes. Expert sequences are single tools. `reroute`, for example, removes pours and autorouted

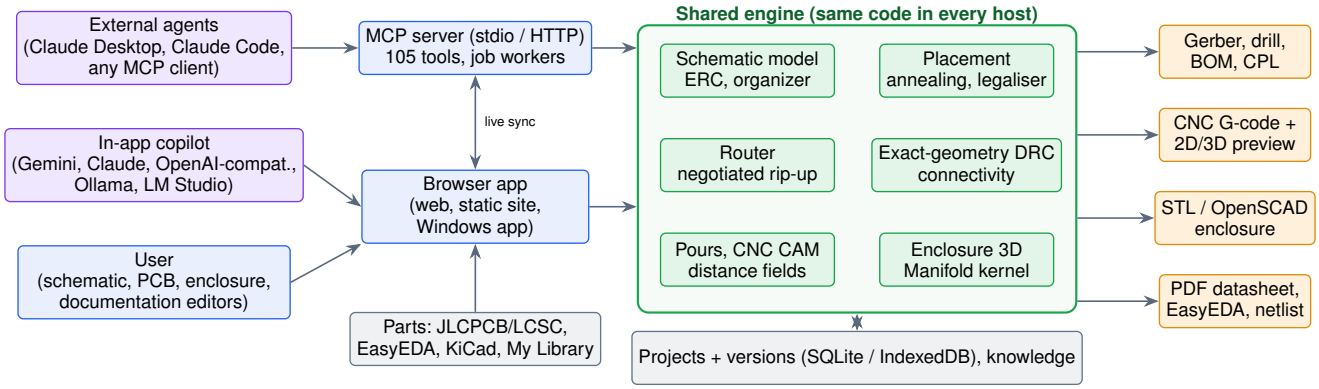


Fig. 1: CircuitPilot architecture. Agents (purple) and the user act through the MCP server or the browser app (blue). Both call the same engine (green), which produces fabrication outputs (orange). Long operations run in worker threads; projects are versioned, so an agent's change can be undone.

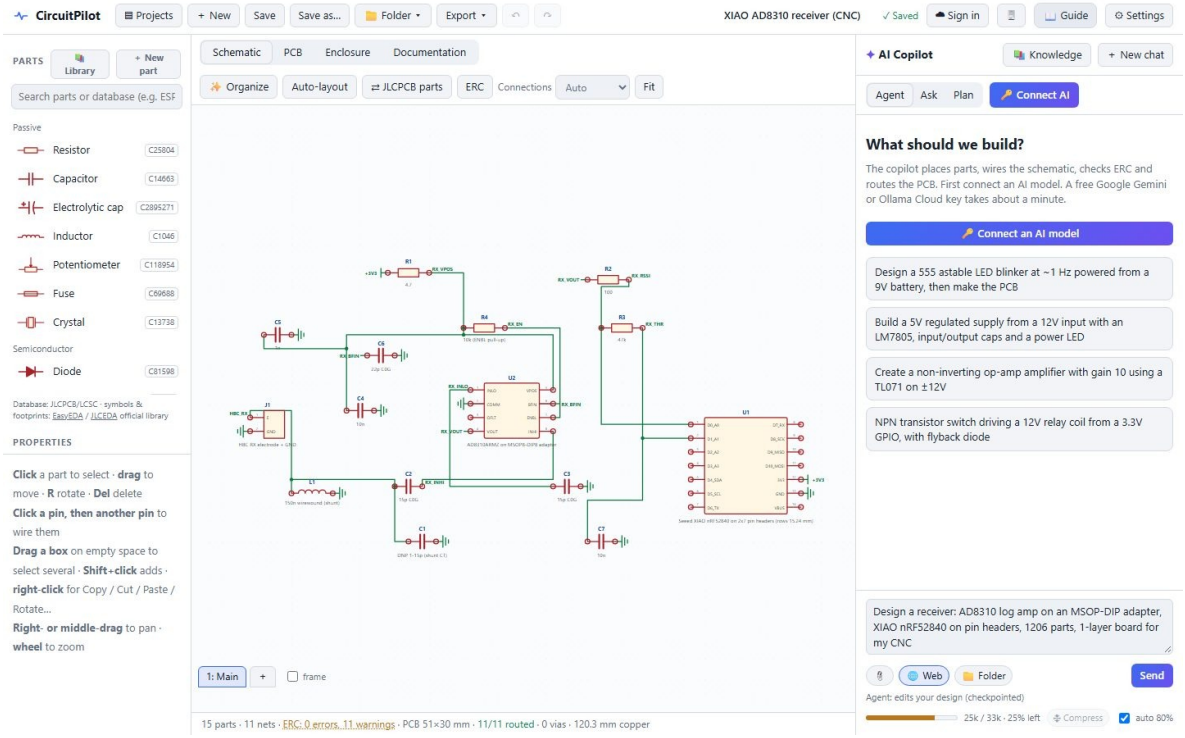


Fig. 2: The CircuitPilot web application: parts library (left), schematic editor (centre) and AI copilot (right). The same editors serve the PCB, enclosure and documentation views.

copper, routes the power nets first, then the rest, restores the pours, adds stitching vias, tidies the silkscreen and runs DRC.

P4 Background jobs. Operations that take minutes return a job identifier, so the MCP session never times out.

P5 Version safety. Every save is versioned and the copilot checkpoints each turn, so a wrong agent action can be undone.

P6 Visual self-check. `view_design` returns a rendered image of the schematic, PCB, 3D board or enclosure, so a multimodal agent can inspect its own result.

Fig. 3 shows the typical agent loop these tools support.

IV. Algorithms

A. Placement

`generate_pcb` places connectors on the nearest board edge (USB, jacks and RF facing outward), then places the remaining parts by a connectivity-driven constructive pass followed by simulated annealing on half-perimeter wire length (HPWL), with the courtyard spacing kept by a legaliser. `optimize_pcb` repeats annealing with extra room around the parts of failing nets, routes each candidate, and keeps the best result. It can also shrink the board step by step while every net still routes.

TABLE II: The 105 MCP tools by category (examples).

Category	#	Examples
Project, versions, jobs	14	`open_project`, `restore_project_version`, `job_status`, `batch`
Parts, knowledge, web	19	`search_parts`, `get_part`, `create_part`, `knowledge_search`
Schematic	13	`add_components`, `connect`, `run_erc`, `organize_schematic`
Board and placement	14	`generate_pcb`, `place_in_free_area`, `optimize_pcb`
Routing and copper	16	`route_pcb`, `reroute`, `add_via`, `connect_pads_to_pour`
Rules and verification	5	`set_design_rules`, `run_drc`, `view_design`
Silkscreen, documentation	8	`arrange_silk_labels`, `set_documentation`
Enclosure	9	`set_enclosure_script`, `check_enclosure`
Fabrication and quotes	7	`export_gerbers`, `export_cnc_gcode`, `jlcpcb_quote`

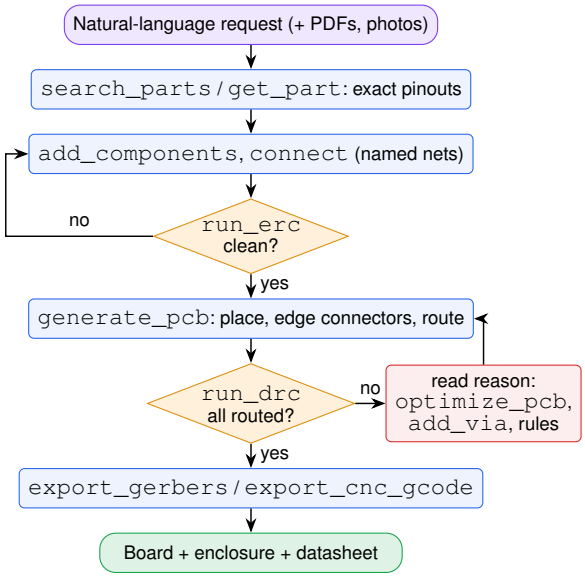


Fig. 3: Typical agent loop. ERC and DRC gate each stage; a failed route returns a reason that maps to a next tool rather than a bare "unrouted".

B. Negotiated-Congestion Grid Router

Grid and obstacles. The board is rasterised at pitch $g = \min(0.25, \max(0.05, \lfloor m/0.05 \rfloor\, 0.025))$ mm, where m is the smallest trace width or clearance; g is increased until the grid has at most 1.5×10^6 cells per layer. For each trace width w, a cell is blocked for a net if its centre lies within $w/2 + c + \delta$ of copper belonging to another net, where c is the applicable clearance (pad, via, hole or edge) and $\delta = \max(0.01, 0.1g)$ mm is a safety margin against rounding. Separate via maps enforce via-to-copper and hole-to-hole clearance. The obstacle maps hold counts that are updated incrementally when copper is added or ripped up, so the map for one net is "counts minus its own contribution", built in a single linear pass.

Search. Each two-terminal connection is a weighted A* [23] search over (layer, cell) states with eight neighbours (four in 90° mode). The cost of a step from cell u to v is

$$c(u, v) = \ell(u, v) + \tau\,[\text{turn}] + h_v + \sigma\,[\text{soft} \wedge v \in \mathcal{O}], \qquad (1)$$

with step length $\ell \in \{1, \sqrt{2}\}$, turn penalty $\tau = 0.6$, history cost h_v, and soft-mode penalty $\sigma = 3$ for crossing copper of other routed nets ($\mathcal{O}$). A layer change costs $\max(8, 2.5/g)$. The heuristic is the octile distance to the nearest target pad, weighted by 1.15. The search runs first inside the bounding box of the tree and the target, enlarged by 8 mm, and then on the whole board. On large windows a bidirectional breadth-first flood from the source tree and the target first tests whether a path exists at all; because a blocked terminal is usually boxed in, this rejects hopeless searches in far fewer expansions than A*.

Multi-terminal nets. Pads already joined by kept copper form islands. A net grows like Prim's algorithm: it starts from the largest island nearest the centroid and repeatedly connects the nearest remaining island to the whole current tree. When a connection does not fit at the net's width, narrower widths are tried if neck-down is allowed. Each committed path is straightened: from every point the farthest point reachable by one clear 0/45/90° segment, or by a two-leg elbow, replaces the grid staircase, and the cells are re-rasterised from the straightened geometry.

Negotiation. Algorithm 1 summarises the rip-up loop. It differs from PathFinder [27] in two ways. First, *blocker discovery*: a failed net is routed once in soft mode, where other nets' copper is passable at a cost, which reveals the specific nets in its way. Second, *selective rip-up*: only the two to four cheapest blockers are removed, ranked by pad count, HPWL and how often they have already been ripped. The probe path also raises the history cost of its corridor, so later searches by other nets avoid it. Moves are accepted when the score $S = 10^7 F + L + 1.5V$ improves (F failed nets, L track length, V vias), and equal-failure moves are accepted with probability 0.3 to escape plateaus. After 25 moves without improvement the router restarts from the best state with the failed nets ordered first.

Exact-geometry closure. The grid is an approximation, so two passes follow. Connectivity is recomputed from exact geometry (tracks, vias and pours), and any net found broken—typically a pour split into islands—is routed again with the live pour as part of its source tree. Then DRC is run, nets with clearance violations are re-routed with an extra 0.04 mm margin, and the repair is kept only if it does not lose connectivity. Nets still open receive the explanation described in principle P2.

C. Copper Pours and SMD Pad Relief

A pour is filled on a raster as a signed distance field: obstacles (other nets' copper grown by the pour clearance, keep-outs, board-edge clearance, drill holes) are subtracted, small or orphaned islands are removed, and the boundary is traced into polygons. With `smdPadRelief`, each SMD pad

Algorithm 1 Negotiated rip-up and reroute with blocker discovery

Input: nets $\mathcal{N}$, placement, rules, time budget T
Output: tracks, vias, open nets with reasons
build obstacle-count maps; $h \leftarrow 0$
route all nets in ascending HPWL order; $B \leftarrow$ snapshot
if some nets failed **then** rip all; route failed nets first; keep the better result
end if
while failed nets exist **and** time $< T$ **do**
  pick a failed net f at random; rip f
  $(P, \mathcal{K}) \leftarrow$ RouteSoft(f) ▷ path, blockers
  $h_v \leftarrow \min(8, h_v + 0.4)$ for v in the halo of P
  if $P = \varnothing$ **then** mark f hard (or allow via-in-pad); **continue**
  end if
  $\mathcal{K} \leftarrow$ 2–4 cheapest of $\mathcal{K}$ (pads, HPWL, rip count)
  rip $\mathcal{K}$; route f; route $\mathcal{K}$ (most-ripped first)
  retry any net of $\mathcal{K} \cup \{f\}$ still failing
  if S improved **or** (no new failure **and** rand < 0.3) **then** accept; update B if best
  else restore
  end if
  if 25 moves without improvement **then** restore B; restart with failed nets first
  end if
end while
restore B; reconnect nets broken in exact geometry (pour islands)
DRC repair with $+0.04$ mm margin; explain open nets

of the pour net is surrounded by a clearance gap crossed by four spokes, so a hand-soldered pad does not sink its heat into the plane. On a CNC-milled board this also leaves a visible solder spot on each pad instead of plain copper.

D. Desktop-CNC Isolation Milling

Algorithm 2 converts the routed single-sided board into G-code for a GRBL desktop mill without an intermediate Gerber or CAM tool. Copper is painted on a fine raster ($g = 0.02$ mm in our runs), and an exact Euclidean distance transform [40] gives the distance from every node to the copper. Isolation rings are iso-contours of this field at $d_k = \phi/2 + k\,\phi(1 - o)$, where ϕ is the groove width of the V-bit at the chosen depth ($\phi = t + 2z\tan(\alpha/2)$ for tip t, depth z and angle α) and o is the overlap between passes. Contours are extracted by oriented marching squares [41], simplified, and oriented for conventional milling. Before writing files, the generator *verifies the toolpath*. It labels the connected copper regions, then labels the copper grown to the narrowest groove the bit could cut; two nets in one grown region would be shorted. Labelling the copper remaining after the widest groove detects nets that the cutter would sever. Copper that the pour leaves out (under SMD parts, in keep-outs) is cleared by zig-zag passes so that no floating slivers remain. Drill files are grouped by bit size, and the board cutout uses an end mill with holding tabs. The same toolpaths are rendered as the milled result in 2D and as a textured, bump-mapped 3D model, so the user can inspect the board before cutting copper.

E. Schematic Organizer and Wire Router

Agents place symbols poorly: they put parts in arbitrary grid positions and connect them with long crossing wires or

Algorithm 2 Verified isolation toolpaths

Input: copper of the milled layer, V-bit (t, α, z), passes K, overlap o
Output: isolation, drill and cutout G-code; 2D/3D previews
$\phi \leftarrow t + 2z\tan(\alpha/2)$; mirror if parts go on the bare side
paint copper mask C and board mask on a grid of pitch g
$D \leftarrow \mathrm{EDT}(C)$ ▷ exact Euclidean distance
for $k = 0 \ldots K-1$ **do** $R_k \leftarrow$ MarchingSquares($D - \phi/2 - k\phi(1-o)$)
end for
$L_C \leftarrow$ Label(C); $L_n \leftarrow$ Label($D \le \phi_{\min}/2$)
if a region of L_n holds two nets **then** report short
if a net splits after the widest groove **then** report cut net
clear pour-free areas (zig-zag); simplify and order paths
emit G-code: isolation, drills per bit, cutout with tabs

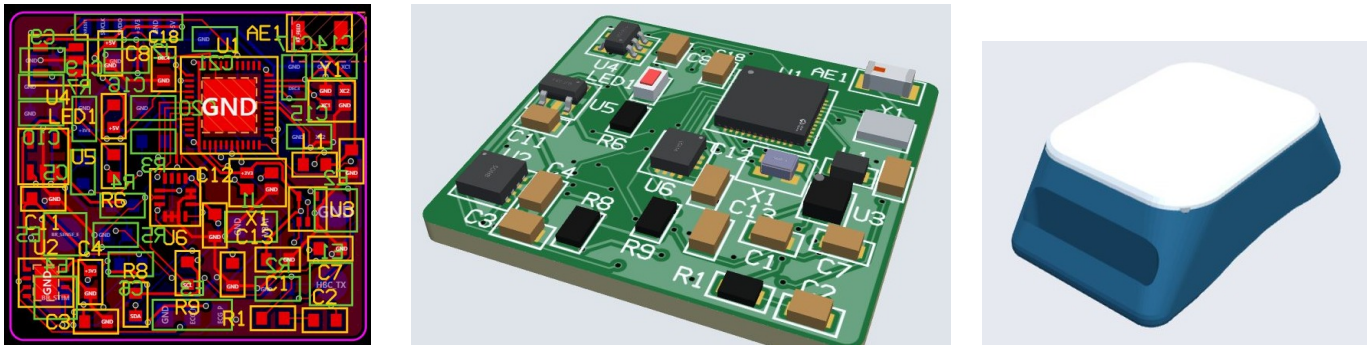
Algorithm 3 Schematic organizer

Input: symbols, nets
Output: positions, rotations, orthogonal wires
main parts $M \leftarrow$ ICs and modules, chained by shared nets
owner(p) $\leftarrow$ BFS from M over signal nets (paired 2-pin parts first)
for each small part p in BFS order **do**
  rotation by role: GND $\downarrow$, supply $\uparrow$, series $\leftrightarrow$
  place beside owner pin; candidates scored by outward $+1.6\cdot$lateral
end for
lay clusters out left to right (wrap); centre
MazeRoute(nets by span): Dijkstra over (cell, dir), bend 4, cross 8

with net labels that hide the circuit's structure. The organizer (Algorithm 3) rebuilds the drawing in the style of a hand-drawn schematic. Integrated circuits and modules form a row; every small part is assigned by breadth-first search over signal nets to the host pin it serves, and is placed beside that pin, continuing outward along chains. Rotation follows conventions: a part on ground stands vertically with ground at the bottom, a part on a supply stands vertically with the supply on top, series parts lie horizontally, and decoupling capacitors attach to supply-named pins. Wires are then routed by a Dijkstra search [21] on a 10-unit grid whose state is (cell, direction): each step costs 1, a bend costs 4, crossing another net costs 8, and running along another net's wire, through a symbol, a power symbol or a net label, or through an unconnected pin end is forbidden. Junction dots appear where three or more wire ends meet, and each net name is placed on its longest horizontal run. This follows the orthogonal-drawing tradition [36] with a router in place of an embedding.

F. Enclosure

An enclosure is either a parametric box or a script in a small modelling language evaluated by the Manifold geometry kernel [42] (WebAssembly), with a BSP fallback. It is built around the board's 3D model, with cut-outs for edge connectors, standoffs at mounting holes, and printability checks (wall thickness, overhangs), and it exports STL and OpenSCAD. 3D views use three.js [43].



(a) Routed board (top red, bottom blue) (b) 3D view with supplier 3D models (c) Generated wearable enclosure

Fig. 4: End-to-end results: a 26×24 mm nRF52832 sensor node (49 parts, 37/37 nets routed) and a 3D-printable enclosure for the HBC wearable node.

TABLE III: Benchmark designs (from the HBC/ECG wearable project).

ID	Design
D1	XIAO_AD8310_RX_CNC_1206_wide
D2	XIAO_nRF52840_Carrier_CNC_1206
D3	HBC_Flex_Electrode_Leads
D4	TXOnly_ECG_OptionB
D5	TXOnly_ECG_OptionC_Small
D6	XIAO_Size_RX_ECG
D7	TXRX_ECG_OptionC
D8	ESP32C3_TXRX_ECG_IMU
D9	RP2040_TXRX_ECG_IMU
D10	Custom_nRF52832_TX_OptionC_ECG_IMU
D11	ESP32C3_WROOM02U_TXRX_ECG_IMU
D12	HBC Wearable Node (TX option A)
D13	Custom_RP2040_TX_OptionC_ECG_IMU
D14	EXTAFE_IMU_ECG_Memory_modes_v2_noViaUnderU1
D15	HBC_Test_Modules_Panel_1206

V. Evaluation

Setup. We selected 15 distinct boards from the project database (Table III), covering single-sided CNC boards with 0.9 mm tracks, a 120 mm flexible electrode carrier, compact two-layer wearable nodes with 0.2 mm tracks around nRF52, ESP32-C3 and RP2040 devices, and a 101-part test panel. The placements are those of the stored designs, made by the agent or the user. For each board, all autorouted copper was removed (hand-drawn copper kept) and `route_pcb` was run headless with a 60 s budget for the negotiation loop (the exact-geometry closure passes may add up to about a minute on boards that do not converge), in the same engine code as the product, once with negotiated rip-up and once with a single pass plus the second ordered pass but no rip-up loop (the ablation). Experiments ran on an Intel Xeon E5-2660 v4 at 2.0 GHz under Node.js 22, single-threaded. DRC uses the exact-geometry checker.

Routing. Table IV and Fig. 5 give the results. The full router completed 11 of 15 boards and 455 of 468 nets; without rip-up, 7 boards and 444 nets were completed. The single pass suffices on the sparse single-sided and small boards, where both variants produce identical copper. The negotiation loop matters on the dense two-layer wearables, where the first pass routinely leaves one to three nets blocked and the loop recovers them at the cost of a few extra vias. Run time grows with board density more than with net count: the single-sided CNC boards route in about a second, the 0.05 mm-grid wearables in 10–50 s, and boards that do not converge consume the whole budget. All fifteen stored designs are fully routed: the four boards left open here (D10, D12–D14) were completed interactively through longer runs, `optimize_pcb` placement changes and, for D10, D13 and D14, a few hand-drawn or locked tracks that the router then keeps. D3's copper is entirely hand-drawn (a flexible electrode carrier), so it only checks that kept copper is respected.

Remaining errors. The DRC errors in Table IV are of two kinds. "Unrouted" entries are nets the router could not complete; for each, it returned the blocked pads and a suggestion, which in interactive use leads the agent to `optimize_pcb` or to a placement change. "Via-in-pad" entries come from vias allowed as a last resort when a pin is boxed in. We report them as errors because the strict rule set forbids them, and the Gerber export refuses such a board until they are fixed.

Fabrication on a desktop CNC. Table V lists the isolation-milling jobs for the two single-sided boards, with and without SMD pad relief, for a 30° V-bit with a 0.1 mm tip at 0.2 mm depth (0.207 mm groove), two passes, 1.0 mm drills and a 2 mm end mill for the cutout. The verification found no short at the narrowest groove and no cut net at the widest groove in any job. Pad relief added 21 rings to D1 (each relieved pad gains its own ring) for about one extra minute of milling, and changed nothing on D2, which has no SMD pads on its pour. Fig. 7 shows the toolpaths and both previews; the D1 board was milled on a CNC 3018 desktop machine.

How much state does an agent read? Fig. 8 compares the size of tool answers with the raw project JSON (about four characters per token). Across the 15 designs, `get_pcb_layout` returns 7 % and `get_design` 15 % of the raw document; the largest board's layout answer is about 1,900 tokens against 29,500 for the raw project. The track list is the exception (50 %), which is why it is filtered by default (`near`, `region`, `vias_only`). The tool definitions themselves amount to about 17,400 tokens—a fixed cost that MCP clients pay once per session.

Schematic organizer. Fig. 6 shows D1's schematic as first placed by the agent and after one click of *Organize*. Measured on the wires the router draws, total wire length fell from 3580

TABLE IV: Routing results. “Single pass” disables the rip-up loop (ablation). Nets: multi-pin nets; pins: connected pins. Bold: all nets routed. Vias, length, time and DRC errors are for the full router.

ID	Parts	Nets	Pins	Layers	Board (mm)	Single pass	Rip-up (ours)	Vias	Length (mm)	Time (s)	DRC err.
D1	15	11	37	1	51×30	**11/11**	**11/11**	0	120	0.6	0
D2	17	15	39	1	52×32	**15/15**	**15/15**	0	132	1.4	0
D3	10	10	20	1	120×62	**10/10**	**10/10**	0	600	1.3	0
D4	20	17	54	2	24×20.4	**17/17**	**17/17**	20	200	11.9	0
D5	22	18	61	2	23.4×20	**18/18**	**18/18**	21	221	11.6	0
D6	32	26	85	2	19.6×23.2	**26/26**	**26/26**	30	287	9.2	1
D7	35	27	94	2	24×24	26/27	**27/27**	43	401	25.2	0
D8	44	32	122	2	25.5×26.2	31/32	**32/32**	54	433	21.6	0
D9	44	34	124	2	26.8×24.3	33/34	**34/34**	54	487	26.3	0
D10	49	37	150	2	26×24	31/37	33/37	84	648	92.5	5
D11	52	36	156	2	25×23	33/36	**36/36**	72	702	48.6	0
D12	55	32	136	2	23.8×30.5	30/32	31/32	57	611	83.4	1
D13	56	45	187	2	32×28	36/45	38/45	108	825	101.4	7
D14	56	57	235	2	56.9×55.8	56/57	56/57	294	2386	113.4	1
D15	101	71	272	2	99.4×89	**71/71**	**71/71**	88	1637	31.9	0
All	608	468	1772			444/468	455/468	925	9690	580.0	15

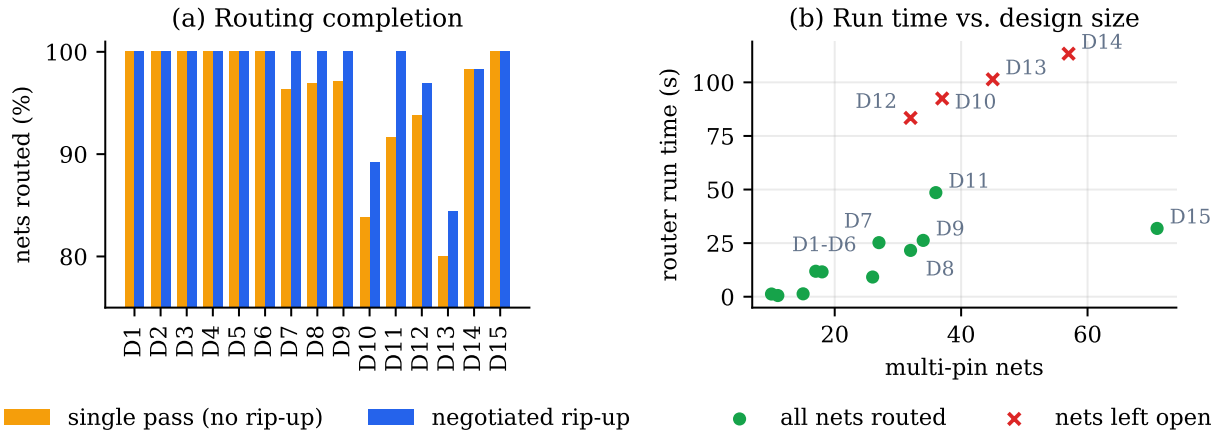
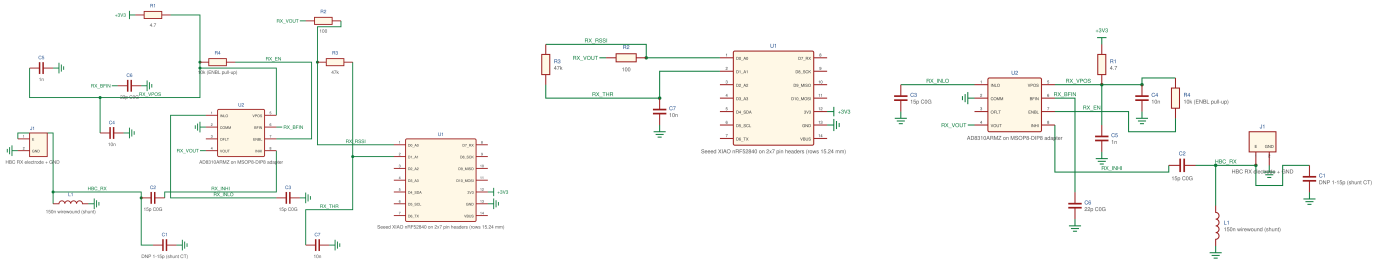


Fig. 5: (a) Share of nets routed per design without and with the negotiated rip-up loop. (b) Router run time against design size for the full router; red crosses mark designs that kept open nets after the 60 s negotiation budget and the closure passes.



(a) As placed by the agent

(b) After *Organize*: parts beside their pins, ground down, supply up

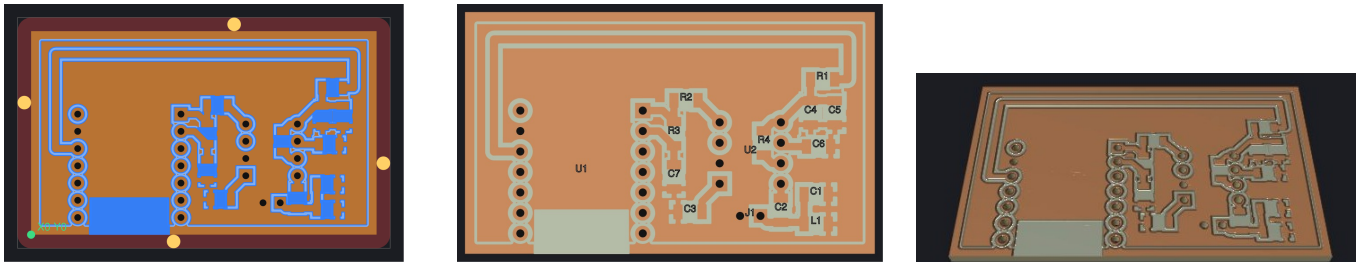
Fig. 6: Schematic of D1 (AD8310 receiver) before and after the organizer.

TABLE V: Isolation-milling jobs (GRBL G-code). Check: shorts at the narrowest groove / nets cut at the widest groove.

ID	Relief	Board (mm)	Rings	Path (mm)	Holes	Lines	Min	Check	Gen. (s)
D1	off	51×30	48	2627	24	5138	40	0/0	5.7
D1	on	51×30	69	2712	24	5775	41	0/0	5.7
D2	off	52×32	61	4138	17	6065	55	0/0	8.1
D2	on	52×32	61	4142	17	6075	55	0/0	7.9

to 2560 units (-28%) and bends from 36 to 27 (-25%), while crossings rose from 2 to 4: the organizer pulls each part next to its pin, which sometimes forces a crossing that a sparser layout avoided. Organizing and routing took 13–416 ms across the 15 designs (54 ms for D1), fast enough to rerun on every edit. Most stored wearable designs use net labels instead of wires; organizing them converts labels to drawn wires, which makes them longer by construction, so we do not report a “before” figure for those.

End-to-end case study. Fig. 2 shows the application with the copilot, and Fig. 4 shows a 49-part nRF52832 sensor node on a 26×24 mm two-layer board (37 nets, 101 vias, fully routed) and the wearable enclosure generated for the



(a) Toolpaths: isolation (blue), cutout (red), tabs (yellow)

(b) Result preview (2D)

(c) Result preview (3D)

Fig. 7: Desktop-CNC output for D1 (AD8310 log-amplifier HBC receiver on a XIAO nRF52840), generated directly from the routed board without an external CAM tool. Through-hole parts go in from the bare side, so the job is mirrored automatically.

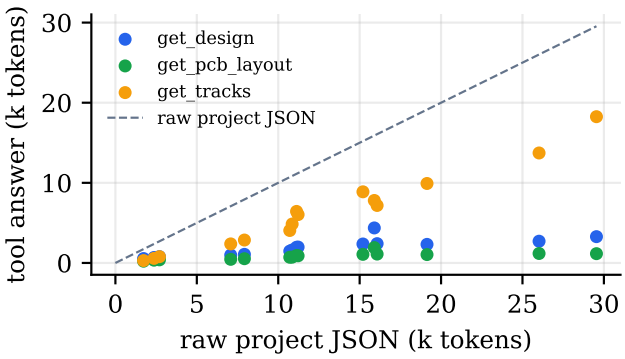


Fig. 8: Size of tool answers against the raw project JSON for the 15 designs. Points below the dashed line save context; the layout summary stays nearly flat as designs grow.

HBC node. Starting from a natural-language request, the agent selected parts with exact LCSC pinouts, wired the schematic, generated and routed the board, and produced Gerber, BOM and pick-and-place files for assembly at JLCPCB, an STL enclosure for multi-jet-fusion printing and a product datasheet. Where the user disagreed with a decision, they changed it through the same tools, and the agent continued from the edited design.

VI. Discussion and Limitations

What the split buys. Because geometry comes only from engines, an agent's mistakes are confined to intent: a wrong part, a wrong connection, a poor rule choice. These are visible in ERC, DRC and the rendered views, and they are reversible through versions. In the other direction, engine failures are worded for the agent, so a failed route becomes a next action instead of a dead end.

Limitations. (i) The router is grid-based and limited to two copper layers; it has no length matching, differential pairs or impedance control, so it targets low-speed mixed-signal boards rather than high-speed designs. (ii) Its search is randomised within a time budget, so dense boards may finish differently between runs; the reported numbers are from single runs. (iii) The benchmark comes from one project family and one designer, and the placements were made in the loop with the tool, which may favour it; a comparison with Freerouting on exported DSN files, and placement-from-scratch experiments, are future work. (iv) We have not run a controlled user study; claims about accessibility are from use, not from measurement. (v) The organizer improves wire length and bends but can add crossings. (vi) Agent behaviour depends on the model used; we did not benchmark models against each other here. (vii) CircuitPilot is source-available under the PolyForm Strict 1.0.0 licence (noncommercial use, no modification); versions up to v2026.10.30.6 remain Apache-2.0.

Future work. Model-to-model comparisons on a shared task set, integration with public PCB benchmarks [16], [17], a user study with non-specialist researchers, and routing beyond two layers.

VII. Conclusion

CircuitPilot shows that a language-model agent can carry a design from a sentence to a manufacturable board when the work is divided correctly: the model decides, and deterministic, explainable engines produce and verify the geometry. A single engine shared by the browser, a static site, a desktop application and an MCP server gives agents 105 tools with compact answers and actionable failures. On 15 real boards, negotiated rip-up completed 11 boards against 7 for a single pass, and desktop-CNC jobs were generated with verified toolpaths and previewed before milling. The tool is available at https://pcbgo.site.

Availability

The application runs in the browser at https://pcbgo.site; source code, the benchmark scripts (`docs/paper/bench`) and a 33-page tutorial are at https://github.com/Abdelhayali/CircuitPilot.

Acknowledgment

Parts of the software and of this manuscript were developed with the assistance of AI coding and writing tools. The author reviewed all content and is responsible for it.

References

[1] KiCad Developers, "KiCad EDA: Schematic capture & PCB design software," accessed: Oct. 2026. [Online]. Available: https://www.kicad.org/

[2] "FlatCAM: PCB prototyping CAD/CAM," accessed: Oct. 2026. [Online]. Available: https://flatcam.org/

[3] pcb2gcode contributors, "pcb2gcode: Command-line tool for isolation, routing and drilling of PCBs," accessed: Oct. 2026. [Online]. Available: https://github.com/pcb2gcode/pcb2gcode

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.

[5] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 68 539–68 551.

[6] K. Chang, Y. Wang, H. Ren, M. Wang, S. Liang, Y. Han, H. Li, and X. Li, "ChipGPT: How far are we from natural language hardware design," arXiv preprint arXiv:2305.14019, 2023. [Online]. Available: https://arxiv.org/abs/2305.14019

[7] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "VeriGen: A large language model for Verilog code generation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 3, pp. 1–31, 2024.

[8] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang *et al.*, "ChipNeMo: Domain-adapted LLMs for chip design," arXiv preprint arXiv:2311.00176, 2023. [Online]. Available: https://arxiv.org/abs/2311.00176

[9] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-Chat: Challenges and opportunities in conversational hardware design," in *Proc. ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, 2023, pp. 1–6.

[10] R. Zhong, X. Du, S. Kai, Z. Tang, S. Xu, H.-L. Zhen, J. Hao, Q. Xu, M. Yuan, and J. Yan, "LLM4EDA: Emerging progress in large language models for electronic design automation," arXiv preprint arXiv:2401.12224, 2023. [Online]. Available: https://arxiv.org/abs/2401.12224

[11] J. Pan, G. Zhou, C.-C. Chang, I. Jacobson, J. Hu, and Y. Chen, "A survey of research in large language models for electronic design automation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 3, pp. 1–21, 2025.

[12] Y. Lai, S. Lee, G. Chen, S. Poddar, M. Hu, D. Z. Pan, and P. Luo, "AnalogCoder: Analog circuit design via training-free code generation," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 1, pp. 379–387, 2025.

[13] R. Matsuo, S. Uhlich, A. Venkitaraman, A. Bonetti, C.-Y. Hsieh, A. Momeni, L. Mauch, A. Capone, E. Ohbuchi, and L. Servadei, "Schemato – an LLM for netlist-to-schematic conversion," in *Proc. ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*, 2025, pp. 1–7.

[14] Q. Luo, R. Ma, X. Zhang, and L. Qiu, "SchGen: PCB schematic generation with semantic-grounded code representations," arXiv preprint arXiv:2605.30345, 2026. [Online]. Available: https://arxiv.org/abs/2605.30345

[15] S. Srinivasan, B. Tan, B. Turnbull, and H. Pearce, "PCB-QA: Evaluating LLMs over the first printed circuit board design question-answer dataset," arXiv preprint arXiv:2606.23704, 2026. [Online]. Available: https://arxiv.org/abs/2606.23704

[16] H. Song, J. Park, W.-S. Choi, S. Bae, H.-S. Jeong, Y. Park, and S. Lee, "PCBWorld: A benchmark environment for engine-grounded PCB design automation," arXiv preprint arXiv:2607.05915, 2026. [Online]. Available: https://arxiv.org/abs/2607.05915

[17] T. Lu, K. Lin, M. Wang, H. Ye, R. Liu, Y. Tian *et al.*, "OmniLayout: A schematic-coupled multimodal benchmark for constraint-aware geometric reasoning in PCB layout," arXiv preprint arXiv:2607.03261, 2026. [Online]. Available: https://arxiv.org/abs/2607.03261

[18] Anthropic, "Introducing the Model Context Protocol," Anthropic News, Nov. 25, 2024, Nov. 2024, accessed: Oct. 2026. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[19] Model Context Protocol, "Model context protocol specification, revision 2026-07-28," 2026, accessed: Oct. 2026. [Online]. Available: https://modelcontextprotocol.io/specification/latest

[20] C. Y. Lee, "An algorithm for path connections and its applications," *IRE Transactions on Electronic Computers*, vol. EC-10, no. 3, pp. 346–365, 1961.

[21] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[22] F. O. Hadlock, "A shortest path algorithm for grid graphs," *Networks*, vol. 7, no. 4, pp. 323–334, 1977.

[23] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[24] D. W. Hightower, "A solution to line-routing problems on the continuous plane," in *Proc. 6th Annual Design Automation Conference (DAC)*, 1969, pp. 1–24.

[25] W. A. Dees and P. G. Karger, "Automated rip-up and reroute techniques," in *Proc. 19th Design Automation Conference (DAC)*, 1982, pp. 432–439.

[26] R. Nair, "A simple yet effective technique for global wiring," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 6, no. 2, pp. 165–172, 1987.

[27] L. McMurchie and C. Ebeling, "PathFinder: A negotiation-based performance-driven router for FPGAs," in *Proc. 3rd ACM International Symposium on Field-Programmable Gate Arrays (FPGA)*, 1995, pp. 111–117.

[28] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI Physical Design: From Graph Partitioning to Timing Closure*. Dordrecht, The Netherlands: Springer, 2011.

[29] Freerouting contributors, "Freerouting: Advanced PCB auto-router," accessed: Oct. 2026. [Online]. Available: https://github.com/freerouting/freerouting

[30] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nova, J. Pak, A. Tong, K. Srinivasa, W. Hang *et al.*, "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021.

[31] L. Vassallo and J. Bajada, "Learning circuit placement techniques through reinforcement learning with adaptive rewards," in *Proc. Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024, pp. 1–6.

[32] Y. He, H. Li, T. Jin, and F. S. Bao, "Circuit routing using Monte Carlo tree search and deep reinforcement learning," in *Proc. International Symposium on VLSI Design, Automation and Test (VLSI-DAT)*, 2022, pp. 1–5.

[33] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, security threats, and future research directions," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 10, pp. 1–37, 2026.

[34] A. Arya, A. Kumar, V. V. Swaminathan, and A. Misra, "Automatic generation of digital system schematic diagrams," in *Proc. 22nd ACM/IEEE Design Automation Conference (DAC)*, 1985, pp. 388–395.

[35] K. Sugiyama, S. Tagawa, and M. Toda, "Methods for visual understanding of hierarchical system structures," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-11, no. 2, pp. 109–125, 1981.

[36] R. Tamassia, "On embedding a graph in the grid with the minimum number of bends," *SIAM Journal on Computing*, vol. 16, no. 3, pp. 421–444, 1987.

[37] A. Sterian, "A closer look at printed circuit board milling," in *Proc. ASEE Annual Conference*, 2001, pp. 6.12.1–6.12.8.

[38] Ucamco, "The Gerber layer format specification, revision 2026.05," 2026, accessed: Oct. 2026. [Online]. Available: https://www.ucamco.com/en/gerber/downloads

[39] Gemini Team, "Gemini: A family of highly capable multimodal models," arXiv preprint arXiv:2312.11805, 2023. [Online]. Available: https://arxiv.org/abs/2312.11805

[40] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance transforms of sampled functions," *Theory of Computing*, vol. 8, pp. 415–428, 2012.

[41] W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3D surface construction algorithm," in *Proc. 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1987, pp. 163–169.

[42] E. Lalish *et al.*, "Manifold: Geometry library for topological robustness," accessed: Oct. 2026. [Online]. Available: https://github.com/elalish/manifold

[43] three.js authors, "three.js: JavaScript 3D library," accessed: Oct. 2026. [Online]. Available: https://threejs.org/